



LABORATORIO PRÁCTICO DE MACHINE LEARNING CON DATOS REALES

CRISP-DM + TDSP + SCRUM ML USANDO DRY BEAN DATASET

Creado por: **Juan Manuel Núñez Velasco**

Dataset real: Dry Bean Dataset, UCI Machine Learning Repository. Fuente:

<https://archive.ics.uci.edu/dataset/602/dry+bean+dataset>

Laboratorio práctico de Machine Learning con datos reales

Tema: Clasificación de variedades de frijol seco usando CRISP-DM, TDSP y Scrum ML

Dataset real: Dry Bean Dataset, UCI Machine Learning Repository

Nivel sugerido: estudiantes de ciencia de datos / IA

Duración estimada: 2 a 4 sesiones de laboratorio

Lenguaje: Python

1. Objetivo del laboratorio

Al finalizar, el estudiante será capaz de:

1. Aplicar **CRISP-DM** para estructurar un proyecto de minería de datos.
2. Usar TDSP para organizar el trabajo técnico como un proyecto reproducible.
3. Trabajar con Scrum ML mediante roles, backlog, sprints y entregables incrementales.
4. Cargar datos reales desde una fuente pública.
5. Explorar, limpiar, preparar y modelar datos tabulares.
6. Entrenar modelos de clasificación con `scikit-learn`.
7. Evaluar modelos con métricas adecuadas.
8. Documentar resultados como si fueran parte de un proyecto profesional.

2. Dataset usado

Se utilizará el **Dry Bean Dataset** del repositorio UCI Machine Learning Repository. El dataset contiene mediciones de granos de frijol seco obtenidas mediante visión por computador.

Fuente oficial: <https://archive.ics.uci.edu/dataset/602/dry+bean+dataset>

Descripción oficial: 13.611 instancias, 16 características y 7 clases de frijol. Las variables describen forma, tamaño y estructura del grano.

Problema de negocio simulado

Una empresa agrícola desea automatizar la clasificación de granos de frijol para mejorar el control de calidad. Actualmente, parte de la clasificación se hace manualmente. El equipo de datos debe construir un modelo que prediga la variedad de frijol a partir de características geométricas extraídas de imágenes.

Variable objetivo

La columna objetivo se llama `Class` y contiene la variedad del frijol.

3. Metodologías que se usarán

3.1 CRISP-DM

CRISP-DM organiza el proyecto en seis fases:

1. Comprensión del negocio: entender el problema y el objetivo.
2. Comprensión de los datos: revisar fuentes, calidad y patrones.
3. Preparación de los datos: limpiar, transformar y separar datos.
4. Modelado: entrenar modelos de Machine Learning.
5. Evaluación: decidir si el modelo cumple el objetivo.
6. Despliegue: preparar el uso del modelo o una demo reproducible.

3.2 TDSP

TDSP, Team Data Science Process, enfatiza:

- Estructura del repositorio;
- Trazabilidad de experimentos;
- Versionado de datos, código y modelos;
- Documentación técnica;
- Reproducibilidad.

3.3 Scrum ML

Scrum ML adapta Scrum a proyectos de Machine Learning. En este laboratorio se trabajará con:

- **Product Owner:** define el valor del modelo.
- **Scrum Master:** ayuda a cumplir el proceso.
- **Equipo de datos:** explora, modela, evalúa y documenta.
- **Backlog:** lista de tareas técnicas y de negocio.
- **Sprints:** ciclos cortos con entregables probables.
- **Definition of Done:** criterios para considerar una tarea terminada.

4. Organización TDSP del proyecto

Crea esta estructura de carpetas:

```
laboratorio_drybean_ml/  
├── data/  
│   ├── raw/  
│   └── processed/  
├── notebooks/  
├── outputs/  
│   ├── models/  
│   └── reports/  
├── src/  
└── README.md
```

Explicación de la estructura

Carpeta	Uso
`data/raw/`	datos originales sin modificar
`data/processed/`	datos limpios o transformados
`notebooks/`	notebooks de exploración
`outputs/models/`	modelos entrenados
`outputs/reports/`	métricas, gráficas y reportes
`src/`	código reutilizable
`README.md`	instrucciones del proyecto

5. Scrum ML: backlog sugerido

Historia de usuario	Tarea técnica	Criterio de aceptación
Como analista, quiero cargar datos reales para analizarlos.	Descargar y leer el dataset.	El dataframe se carga sin errores.
Como científico de datos, quiero revisar la calidad de los datos.	Ver nulos, duplicados y clases.	Hay resumen de calidad.
Como equipo, queremos un modelo base.	Entrenar `LogisticRegression`.	Se reporta accuracy y F1 macro.
Como equipo, queremos mejorar el desempeño.	Entrenar `RandomForestClassifier`.	Se compara contra baseline.
Como usuario final, quiero entender el resultado.	Crear matriz de confusión y reporte.	Se documentan hallazgos.

Como equipo técnico, quiero reproducibilidad.	Guardar modelo con `joblib`.	El modelo se puede cargar y usar.
---	------------------------------	-----------------------------------

Sprint 1: comprensión y preparación

Entregables:

- Dataset cargado;
- Análisis exploratorio básico;
- Separación train/test;
- Pipeline de preprocesamiento.

Sprint 2: modelado y evaluación

Entregables:

- Modelo baseline;
- Modelo mejorado;
- Comparación de métricas;
- Modelo guardado.

6. Preparación del entorno

Ejecuta en una terminal:

```
python -m venv .venv
source .venv/bin/activate
pip install pandas numpy matplotlib scikit-learn ucimlrepo joblib openpyxl
```

En Windows PowerShell:

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install pandas numpy matplotlib scikit-learn ucimlrepo joblib openpyxl
```

Laboratorio paso a paso

Paso 1. Crear un archivo de trabajo

Crea un notebook llamado:

```
notebooks/01_laboratorio_drybean.ipynb
```

También puedes usar un archivo Python:

```
src/laboratorio_drybean.py
```

Paso 2. Importar librerías

Código

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from ucimlrepo import fetch_ucirepo

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score, classification_report,
ConfusionMatrixDisplay

import joblib
```

Explicación línea por línea

Línea	Explicación
`import pandas as pd`	Importa pandas para trabajar con tablas de datos.
`import numpy as np`	Importa NumPy para operaciones numéricas.
`import matplotlib.pyplot as plt`	Importa herramientas para crear gráficas.
`from ucimlrepo import fetch_ucirepo`	Importa una función para descargar datasets oficiales de UCI.
`from sklearn.model_selection import train_test_split`	Permite dividir datos en entrenamiento y prueba.
`from sklearn.preprocessing import StandardScaler`	Estandariza variables numéricas para que tengan escalas comparables.
`from sklearn.pipeline import Pipeline`	Permite encadenar preprocesamiento y modelo en un solo objeto.
`from sklearn.linear_model import LogisticRegression`	Importa un modelo lineal usado como baseline.
`from sklearn.ensemble import RandomForestClassifier`	Importa un modelo de ensamble basado en árboles de decisión.
`from sklearn.metrics import ...`	Importa métricas y herramientas de evaluación.
`import joblib`	Permite guardar y cargar modelos entrenados.

Paso 3. Cargar el dataset real desde UCI

Código

```
dry_bean = fetch_ucirepo(id=602)

X = dry_bean.data.features
y = dry_bean.data.targets

df = pd.concat([X, y], axis=1)

df.head()
```

Explicación línea por línea

Línea	Explicación
`dry_bean = fetch_ucirepo(id=602)`	Descarga el dataset Dry Bean desde UCI usando su identificador oficial.
`X = dry_bean.data.features`	Guarda las variables predictoras en `X`.
`y = dry_bean.data.targets`	Guarda la variable objetivo en `y`.
`df = pd.concat([X, y], axis=1)`	Une variables predictoras y objetivo en una sola tabla.
`df.head()`	Muestra las primeras cinco filas para inspección inicial.

Actividad del estudiante

Responde:

1. ¿Cuántas columnas tiene el dataset?
2. ¿Cuál es la variable objetivo?
3. ¿El problema es de clasificación o regresión?

Paso 4. Comprensión de datos según CRISP-DM

Código

```
print(df.shape)
print(df.info())
print(df.describe())
```

Explicación línea por línea

Línea	Explicación
`print(df.shape)`	Muestra número de filas y columnas.
`print(df.info())`	Muestra tipos de datos y valores no nulos.
`print(df.describe())`	Calcula estadísticas descriptivas de las variables numéricas.

Interpretación esperada

El estudiante debe identificar:

- Número de registros;
- Número de variables;
- Tipos de datos;
- Posibles valores extremos;
- Si hay columnas no numéricas aparte de la clase.

Paso 5. Revisar valores nulos y duplicados

Código

```
missing_values = df.isna().sum()
duplicated_rows = df.duplicated().sum()

print(missing_values)
print("Filas duplicadas:", duplicated_rows)
```

Explicación línea por línea

Línea	Explicación
`missing_values = df.isna().sum()`	Cuenta valores faltantes por columna.
`duplicated_rows = df.duplicated().sum()`	Cuenta filas repetidas completas.
`print(missing_values)`	Muestra el conteo de valores nulos.
`print("Filas duplicadas:", duplicated_rows)`	Muestra cuántas filas duplicadas existen.

Decisión técnica

Si existen duplicados, se pueden eliminar para evitar que el modelo vea observaciones repetidas.

```
df = df.drop_duplicates()
```

Explicación: `drop\_duplicates()` elimina filas idénticas y devuelve un dataframe sin esas repeticiones.

Paso 6. Analizar la variable objetivo

Código

```
class_counts = df["Class"].value_counts()
print(class_counts)

class_counts.plot(kind="bar")
plt.title("Distribución de clases")
plt.xlabel("Clase de frijol")
plt.ylabel("Cantidad de registros")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

Explicación línea por línea

Línea	Explicación
<code>`class_counts = df["Class"].value_counts()`</code>	Cuenta cuántos ejemplos hay por clase.
<code>`print(class_counts)`</code>	Muestra la distribución numérica de clases.
<code>`class_counts.plot(kind="bar")`</code>	Crea un gráfico de barras.
<code>`plt.title(...)`</code>	Agrega título a la gráfica.
<code>`plt.xlabel(...)`</code>	Nombra el eje horizontal.
<code>`plt.ylabel(...)`</code>	Nombra el eje vertical.
<code>`plt.xticks(rotation=45)`</code>	Rota las etiquetas para que se lean mejor.
<code>`plt.tight_layout()`</code>	Ajusta espacios para evitar cortes visuales.
<code>`plt.show()`</code>	Muestra la gráfica.

Pregunta

¿Las clases están balanceadas? Explica cómo esto puede afectar el entrenamiento.

Paso 7. Separar variables predictoras y objetivo

Código

```
X = df.drop(columns="Class")
y = df["Class"]
```

Explicación línea por línea

Línea	Explicación
<code>`X = df.drop(columns="Class")`</code>	Crea la matriz de variables predictoras quitando la columna objetivo.
<code>`y = df["Class"]`</code>	Guarda la columna objetivo que el modelo debe aprender a predecir.

Paso 8. Separar entrenamiento y prueba

Código

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

Explicación línea por línea

Línea	Explicación
<code>`X_train, X_test, y_train, y_test = ...`</code>	Recibe los subconjuntos de entrenamiento y prueba.
<code>`train_test_split(`</code>	Función que divide datos de forma aleatoria.
<code>`X,`</code>	Variables predictoras.

<code>`y`,`</code>	Etiquetas reales.
<code>`test_size=0.2`,`</code>	Reserva el 20% de los datos para prueba.
<code>`random_state=42`,`</code>	Fija una semilla para que el resultado sea reproducible.
<code>`stratify=y`</code>	Mantiene proporciones similares de clases en entrenamiento y prueba.

Conexión con TDSP

La semilla ``random_state=42`` ayuda a la reproducibilidad. En un proyecto real, esta decisión se documenta para que otro equipo pueda repetir el experimento.

Paso 9. Crear modelo baseline con regresión logística

Código

```
baseline_model = Pipeline(steps=[
    ("scaler", StandardScaler()),
    ("classifier", LogisticRegression(max_iter=1000, random_state=42))
])

baseline_model.fit(X_train, y_train)
```

Explicación línea por línea

Línea	Explicación
<code>`baseline_model = Pipeline(steps=[`</code>	Crea una tubería de procesamiento y modelado.
<code>` ("scaler", StandardScaler() ),`</code>	Primer paso: estandariza las variables numéricas.
<code>` ("classifier", LogisticRegression(...) )`</code>	Segundo paso: entrena una regresión logística multiclase.
<code>`max_iter=1000`</code>	Aumenta el número máximo de iteraciones para facilitar la convergencia.
<code>`random_state=42`</code>	Hace reproducible el entrenamiento.
<code>`])`</code>	Cierra la definición del pipeline.
<code>`baseline_model.fit(X_train, y_train)`</code>	Ajusta el pipeline usando los datos de entrenamiento.

¿Por qué baseline?

Un baseline es un primer modelo sencillo. Sirve como punto de comparación. Si un modelo más complejo no mejora al baseline, quizá no vale la pena usarlo.

Paso 10. Evaluar el modelo baseline

Código

```
y_pred_baseline = baseline_model.predict(X_test)

accuracy_baseline = accuracy_score(y_test, y_pred_baseline)
f1_baseline = f1_score(y_test, y_pred_baseline, average="macro")

print("Accuracy baseline:", accuracy_baseline)
print("F1 macro baseline:", f1_baseline)
print(classification_report(y_test, y_pred_baseline))
```

Explicación línea por línea

Línea	Explicación
<code>y_pred_baseline = baseline_model.predict(X_test)</code>	Genera predicciones sobre datos no vistos durante el entrenamiento.
<code>accuracy_baseline = accuracy_score(...)</code>	Calcula proporción de predicciones correctas.
<code>f1_baseline = f1_score(..., average="macro")</code>	Calcula F1 promedio dando peso similar a cada clase.
<code>print("Accuracy baseline:", ...)</code>	Imprime el accuracy.
<code>print("F1 macro baseline:", ...)</code>	Imprime el F1 macro.
<code>print(classification_report(...))</code>	Muestra precisión, recall y F1 por clase.

Pregunta

¿Por qué F1 macro puede ser más informativo que accuracy cuando las clases no están perfectamente balanceadas?

Paso 11. Entrenar modelo mejorado con Random Forest

Código

```
forest_model = RandomForestClassifier(
    n_estimators=200,
    random_state=42,
    class_weight="balanced"
)

forest_model.fit(X_train, y_train)
```

Explicación línea por línea

Línea	Explicación
<code>forest_model = RandomForestClassifier(</code>	Crea un modelo de bosque aleatorio.
<code>n_estimators=200,</code>	Usa 200 árboles de decisión.
<code>random_state=42,</code>	Hace reproducible el entrenamiento.
<code>class_weight="balanced"</code>	Ajusta pesos para compensar diferencias entre clases.
<code>)</code>	Cierra la configuración del modelo.

<code>`forest_model.fit(X_train, y_train)`</code>	Entrena el bosque aleatorio con datos de entrenamiento.
---	---

Nota técnica

Random Forest no necesita escalamiento obligatorio, porque se basa en árboles. Aun así, se compara con el baseline para justificar su uso.

Paso 12. Evaluar Random Forest

Código

```
y_pred_forest = forest_model.predict(X_test)

accuracy_forest = accuracy_score(y_test, y_pred_forest)
f1_forest = f1_score(y_test, y_pred_forest, average="macro")

print("Accuracy Random Forest:", accuracy_forest)
print("F1 macro Random Forest:", f1_forest)
print(classification_report(y_test, y_pred_forest))
```

Explicación línea por línea

Línea	Explicación
<code>`y_pred_forest = forest_model.predict(X_test)`</code>	Predice clases con el modelo Random Forest.
<code>`accuracy_forest = accuracy_score(...)`</code>	Calcula accuracy del modelo.
<code>`f1_forest = f1_score(..., average="macro")`</code>	Calcula F1 macro.
<code>`print(...)`</code>	Muestra métricas generales.
<code>`classification_report(...)`</code>	Muestra métricas por clase.

Paso 13. Comparar modelos

Código

```
results = pd.DataFrame({
    "modelo": ["Logistic Regression", "Random Forest"],
    "accuracy": [accuracy_baseline, accuracy_forest],
    "f1_macro": [f1_baseline, f1_forest]
})

print(results)
```

Explicación línea por línea

Línea	Explicación
<code>`results = pd.DataFrame({`</code>	Crea una tabla para comparar modelos.
<code>`"modelo": [...)`</code>	Define los nombres de los modelos.
<code>`"accuracy": [...)`</code>	Guarda los valores de accuracy.
<code>`"f1_macro": [...)`</code>	Guarda los valores de F1 macro.
<code>`})`</code>	Cierra la creación del dataframe.
<code>`print(results)`</code>	Imprime la tabla comparativa.

Decisión CRISP-DM

En la fase de evaluación se decide qué modelo pasa a despliegue. El criterio recomendado para este laboratorio es:

- Priorizar `f1\_macro`;
- Revisar errores por clase;
- Preferir el modelo más simple si el desempeño es muy parecido.

Paso 14. Matriz de confusión

Código

```
ConfusionMatrixDisplay.from_predictions(  
    y_test,  
    y_pred_forest,  
    xticks_rotation=45  
)  
plt.title("Matriz de confusión - Random Forest")  
plt.tight_layout()  
plt.show()
```

Explicación línea por línea

Línea	Explicación
`ConfusionMatrixDisplay.from_predictions(`	Crea una matriz de confusión a partir de etiquetas reales y predichas.
`y_test`,`	Etiquetas reales del conjunto de prueba.
`y_pred_forest`,`	Etiquetas predichas por Random Forest.
`xticks_rotation=45`	Rota etiquetas del eje horizontal para facilitar lectura.
`)`	Cierra la función.
`plt.title(...)`	Agrega título a la gráfica.
`plt.tight_layout()`	Ajusta el diseño de la figura.
`plt.show()`	Muestra la matriz.

Pregunta

¿Qué clases se confunden más entre sí? Propón una explicación basada en variables de forma o tamaño.

Paso 15. Importancia de variables

Código

```
feature_importance = pd.DataFrame({
    "feature": X.columns,
    "importance": forest_model.feature_importances_
})

feature_importance = feature_importance.sort_values(
    by="importance",
    ascending=False
)

print(feature_importance.head(10))
```

Explicación línea por línea

Línea	Explicación
<code>feature_importance = pd.DataFrame({`</code>	Crea una tabla con variables e importancias.
<code>"feature": X.columns,`</code>	Usa los nombres de columnas como nombres de variables.
<code>"importance":` forest_model.feature_importances_`</code>	Extrae la importancia calculada por Random Forest.
<code>`)`</code>	Cierra la tabla.
<code>feature_importance.sort_values(...)`</code>	Ordena variables de mayor a menor importancia.
<code>by="importance"`</code>	Indica la columna usada para ordenar.
<code>ascending=False`</code>	Orden descendente.
<code>print(feature_importance.head(10))`</code>	Muestra las diez variables más importantes.

Gráfica

```
feature_importance.head(10).plot(
    x="feature",
    y="importance",
    kind="bar"
)

plt.title("Top 10 variables más importantes")
plt.xlabel("Variable")
plt.ylabel("Importancia")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

Explicación breve

La importancia de variables ayuda a explicar el modelo, pero no prueba causalidad. Solo indica qué variables fueron útiles para separar clases dentro del modelo entrenado.

Paso 16. Guardar el modelo entrenado

Código

```
joblib.dump(forest_model, "outputs/models/random_forest_drybean.joblib")
```

Explicación línea por línea

Línea	Explicación
<code>`joblib.dump(...)`</code>	Guarda un objeto Python en disco.
<code>`forest_model`</code>	Es el modelo que se desea guardar.
<code>`"outputs/models/random_forest_drybean.joblib"`</code>	Es la ruta donde quedará guardado el modelo.

Conexión con TDSP

Guardar el modelo permite reproducir resultados, compartir el artefacto con otro equipo o usarlo en una demo.

Paso 17. Cargar el modelo y probar una predicción

Código

```
loaded_model = joblib.load("outputs/models/random_forest_drybean.joblib")

sample = X_test.iloc[[0]]
prediction = loaded_model.predict(sample)

print("Predicción:", prediction[0])
print("Valor real:", y_test.iloc[0])
```

Explicación línea por línea

Línea	Explicación
<code>`loaded_model = joblib.load(...)`</code>	Carga el modelo guardado desde disco.
<code>`sample = X_test.iloc[[0]]`</code>	Selecciona una fila de prueba conservando formato de dataframe.
<code>`prediction = loaded_model.predict(sample)`</code>	Predice la clase de esa muestra.
<code>`print("Predicción:", prediction[0])`</code>	Muestra la predicción individual.
<code>`print("Valor real:", y_test.iloc[0])`</code>	Muestra la etiqueta verdadera para comparar.

Paso 18. Mini despliegue local con función reutilizable

Código

```
def predict_bean_class(model, input_data):
    prediction = model.predict(input_data)
    return prediction[0]

example_prediction = predict_bean_class(loaded_model, sample)
print(example_prediction)
```

Explicación línea por línea

Línea	Explicación
<code>`def predict_bean_class(model, input_data):`</code>	Define una función que recibe un modelo y datos de entrada.
<code>`prediction = model.predict(input_data)`</code>	Usa el modelo para predecir.
<code>`return prediction[0]`</code>	Devuelve la primera predicción.
<code>`example_prediction = ...`</code>	Ejecuta la función con el modelo cargado y una muestra.
<code>`print(example_prediction)`</code>	Muestra el resultado.

Conexión con CRISP-DM

Este paso representa una versión simple de la fase de despliegue: el modelo ya puede recibir datos nuevos y devolver una predicción.

Entregables del estudiante

Entregable 1. Notebook ejecutado

Debe incluir:

- Carga de datos;
- Análisis exploratorio;
- Limpieza básica;
- Entrenamiento de modelos;
- Evaluación;
- Conclusiones.

Entregable 2. Reporte breve

Formato sugerido:

1. Comprensión del negocio: problema y objetivo.
2. Comprensión de datos: tamaño, variables, clases, calidad.
3. Preparación: transformaciones realizadas.
4. Modelado: modelos entrenados.
5. Evaluación: métricas y comparación.
6. Despliegue: cómo se guardó y probó el modelo.
7. Scrum ML: backlog, sprint review y retrospectiva.

Entregable 3. Evidencia Scrum ML

Cada equipo debe entregar:

- Backlog inicial;
- Tablero simple con columnas `Por hacer`, `En progreso`, `Hecho`;

- Tareas realizadas por sprint;
- Retrospectiva con 3 puntos:
 - Qué funcionó;
 - Qué no funcionó;
 - Qué mejorarían.

Definition of Done del laboratorio

Una práctica se considera terminada si:

1. El notebook corre de inicio a fin sin errores.
2. El dataset se descarga o carga correctamente.
3. Se reportan valores nulos y duplicados.
4. Se entrena al menos un modelo baseline y un modelo alternativo.
5. Se comparan modelos con accuracy y F1 macro.
6. Se incluye matriz de confusión.
7. Se guarda el modelo final.
8. Se explica el resultado en lenguaje comprensible.
9. Se documenta la relación con CRISP-DM, TDSP y Scrum ML.

Rúbrica de evaluación

Criterio	Puntaje
Aplica correctamente CRISP-DM	20
Organiza el proyecto con enfoque TDSP	15
Evidencia Scrum ML	15
Código funcional y reproducible	20
Evaluación del modelo	15
Interpretación y conclusiones	15
Total	100

Preguntas finales de reflexión

1. ¿Qué fase de CRISP-DM fue más importante en este laboratorio?
2. ¿Qué ventaja ofrece TDSP frente a trabajar con archivos desordenados?
3. ¿Cómo ayuda Scrum ML cuando el resultado del modelo es incierto?

- ¿Por qué se usa un modelo baseline?
- ¿Qué métrica usarías para escoger el mejor modelo y por qué?
- ¿Qué harías si una clase tiene muchos más errores que las demás?
- ¿Qué información necesitarías antes de llevar este modelo a producción?

Extensiones opcionales

Opción A. Validación cruzada

Usar `cross\_val\_score` para evaluar estabilidad del modelo.

Opción B. Búsqueda de hiperparámetros

Usar `GridSearchCV` para probar valores de `n\_estimators`, `max\_depth` y `min\_samples\_split`.

Opción C. Archivo de predicciones

Guardar predicciones en CSV:

```
predictions_df = X_test.copy()
predictions_df["real_class"] = y_test.values
predictions_df["predicted_class"] = y_pred_forest
predictions_df.to_csv("outputs/reports/predicciones_drybean.csv", index=False)
```

Explicación línea por línea

Línea	Explicación
`predictions_df = X_test.copy()`	Copia los datos de prueba para no modificar el original.
`predictions_df["real_class"] = y_test.values`	Agrega la clase real.
`predictions_df["predicted_class"] = y_pred_forest`	Agrega la clase predicha.
`to_csv(..., index=False)`	Guarda el archivo sin incluir el índice del dataframe.

Plantilla de daily Scrum para el equipo

Cada integrante responde:

- ¿Qué hice desde la última reunión?
- ¿Qué haré ahora?
- ¿Qué bloqueo tengo?

Plantilla de retrospectiva

Pregunta	Respuesta del equipo
¿Qué salió bien?	
¿Qué fue difícil?	
¿Qué cambiaríamos en el siguiente sprint?	
¿Qué aprendizaje técnico obtuvimos?	

Conclusión esperada

Este laboratorio muestra que un proyecto de Machine Learning no consiste solo en entrenar un modelo. También requiere entender el problema, organizar el trabajo, documentar decisiones, evaluar resultados y preparar una forma reproducible de uso. CRISP-DM da la lógica del proyecto, TDSP da orden técnico y Scrum ML ayuda a gestionar el trabajo en equipo.